Chapter 11: Distributed Databases

Section 11.1 — Concepts & Data Distribution Techniques

- Concepts of Distributed Databases (definition, need, characteristics, architecture)
- Data Distribution Techniques — Fragmentation, Replication, Allocation
- Horizontal, Vertical & Hybrid Fragmentation with examples
- Labeled diagrams for every concept
- Comparison tables, Exam Point / Remember / Common Mistake / Shortcut boxes
- Quick Revision Cheat Sheet

Study Notes prepared for OAVS Computer Teacher (TGT) & Competitive Exam Preparation

11.1.A Concepts of Distributed Databases

1. What is a Distributed Database (DDB)?

Definition: A **Distributed Database** is a single logical database whose data is **physically stored across two or more computers (sites)**, which may be located in the same building or spread across different cities/countries, all connected by a computer network. Although the data is stored at multiple sites, the system appears to the user as **one single database**.

Real-world example: A national bank has branches in Delhi, Mumbai, and Kolkata. Each branch stores its own customers' data locally, but a customer can check their balance or transfer money from any branch as if all the data were in one place. This is a Distributed Database.

Distributed Database Management System (DDBMS)

A **DDBMS** is the software that manages a distributed database and makes the distribution **transparent (invisible)** to the user. It coordinates data storage, retrieval, and updates across all the sites.

Exam Point
DDB = Data is distributed. DDBMS = Software that manages the distributed data and hides the distribution from the user.

2. Why Do We Need Distributed Databases?

Need / Reason	Explanation
Reflects organizational structure	Companies have branches/offices in different locations; each site naturally keeps its own local data.
Improved reliability & availability	If one site fails, other sites keep working — the whole system does not go down.
Faster local access	Data used most often at a site is stored at that site, reducing network delay.
Reduced communication cost	Local queries are processed locally instead of always contacting a central server.
Easy expansion (Scalability)	New sites/branches can be added without redesigning the whole system.
Sharing of data	Users at one site can access data stored at another site when needed.

3. Characteristics of a Distributed Database

- **Data is logically related** but **physically distributed** over multiple sites.
- **Distribution Transparency:** the user need not know where the data is physically stored.
- **Local Autonomy:** each site can manage its own local data independently.
- **No reliance on a single central site** (unlike a centralized database).
- Sites are connected through a **communication network**.
- Supports both **local transactions** (data needed is only at one site) and **global/distributed transactions** (data needed is spread across sites).

Remember
The single most important characteristic tested in exams is **"Distribution Transparency"** — the user sees ONE database even though data sits at MANY sites.

4. Basic Architecture of a Distributed Database

A typical distributed database consists of multiple **sites (nodes)**, each running its own local DBMS, connected via a **network**. A layer of software called the **DDBMS** sits on top and coordinates all the sites so that users can run queries that combine data from several sites.

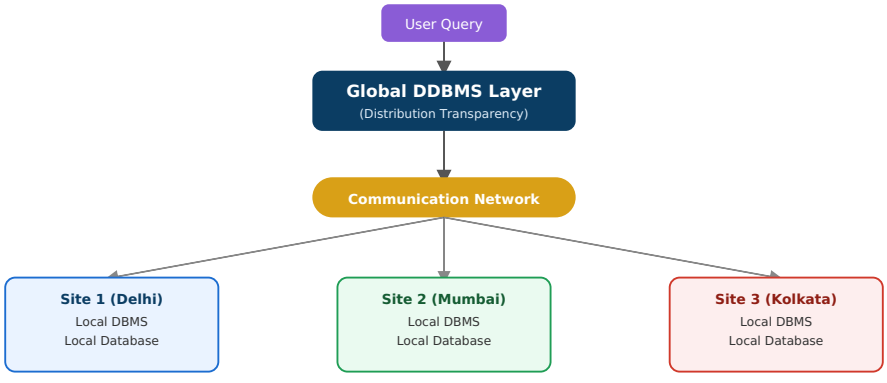


Fig 1: Basic Distributed Database Architecture — the user's query goes to the Global DDBMS layer, which presents one logical database and routes work over the network to the relevant site(s); each site has its own local DBMS and local database.

5. Basic Working of a Distributed Database

1. The user submits a query through any site, without knowing where the actual data is stored.
2. The **global DDBMS** layer breaks the query into sub-queries for the relevant sites (this is possible because it knows the fragmentation/allocation details).
3. Each site's **local DBMS** executes its part of the query on its local data.
4. Partial results from each site travel back over the **network**.
5. The DDBMS **combines (merges)** the partial results and returns one final answer to the user.

> **Shortcut to Remember**
> Think "Query In → Split → Execute Locally → Collect → Combine → Answer Out". This is the working cycle of every distributed database.

6. Data Distributed Over Geographically Separated Sites

The core idea of a DDB is that data does **not sit in one place**. It is spread over sites that may be in different buildings, cities, or even countries, all linked by a network (LAN/WAN/Internet). Each site stores the portion of data that is most relevant to it, while still being part of one integrated system.

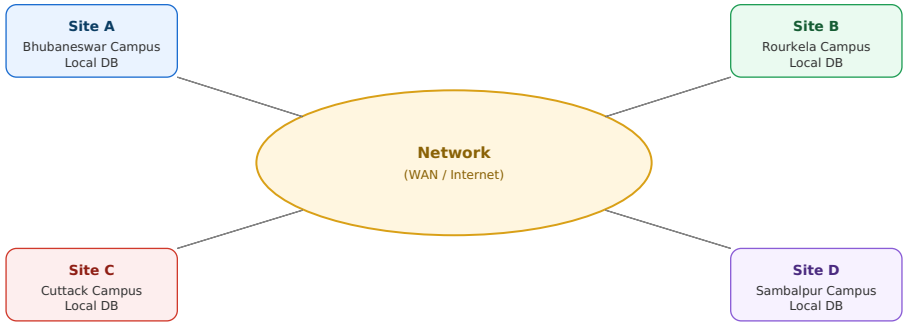


Fig 2: Multiple geographically distributed sites (e.g. university campuses) connected through a network, each holding its own local data as part of one distributed database.

> **Common Mistake**
> Students often confuse a **Distributed Database** with simply "many databases on many computers". The key difference is that a DDB behaves as **ONE logical unit** with coordinated management — random unconnected databases are NOT a distributed database.

> **Important Note**
> A Distributed Database is **not** just "many databases sitting on many computers." The key point is that a DDB works as **ONE logical unit** with coordinated management across all sites — if databases are separate and unconnected, with no coordination between them, they do **NOT** count as a distributed database.

Centralized Database	Distributed Database
All data stored at a single site	Data stored across multiple sites
Single point of failure	Failure of one site does not stop the whole system
Simple to manage	Complex to manage (needs coordination)
Higher network cost for remote users	Faster local access, lower cost for local users
Difficult to scale geographically	Easy to add new sites

11.1.B Data Distribution Techniques

Once we decide to distribute a database, we must decide **how** to break up and **where** to place the data across sites. There are three main techniques used together to design a distributed database:

Technique	Basic Idea
Fragmentation	Dividing a table (relation) into smaller pieces called fragments.
Allocation	Deciding at which site(s) each fragment (or replica) will actually be stored.
Replication	Storing copies of the same data (or fragment) at more than one site.

Exam Point
Order of the design process: **Fragmentation → Allocation → Replication (if needed)**. First decide how to break the table, then decide where each piece goes, and then decide if extra copies are needed.

1. Fragmentation

Definition: Fragmentation is the process of **dividing a single global relation (table)** into smaller logical pieces called **fragments**, which are then stored at different sites. This is done so that each site keeps only the data it needs most often.

Correctness Rules of Fragmentation

- **Completeness:** Every data item of the original relation must appear in at least one fragment (no data should be lost).
- **Reconstruction:** It must be possible to rebuild the original relation from its fragments (usually using UNION or JOIN).
- **Disjointness:** Ideally, a data item should appear in only one fragment (to avoid unnecessary duplication) — except for the key attribute in vertical fragmentation, which is intentionally repeated.

Remember
The three rules — **Completeness, Reconstruction, Disjointness** — are frequently asked directly in exams. Remember them as "**C-R-D**".

(a) Horizontal Fragmentation

Definition: The relation is divided **row-wise**. Each fragment contains a **subset of tuples (rows)** of the original table, but **all its columns**. A condition (WHERE-clause-like predicate) decides which rows go into which fragment. The original relation can be reconstructed by taking the **UNION** of all its horizontal fragments.

Example: A table *STUDENT(RollNo, Name, Branch, City)* can be horizontally fragmented so that: Fragment 1 = rows where City = "Rourkela"; Fragment 2 = rows where City = "Cuttack". Each fragment has the same columns but different rows.

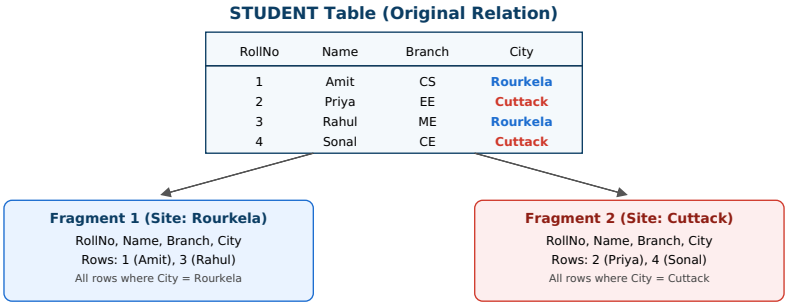


Fig 3: Horizontal Fragmentation — rows are split by a condition (City), each fragment keeps all columns but only some rows; the original table is rebuilt by taking the UNION of all fragments.

Types of Horizontal Fragmentation:

- **Primary Horizontal Fragmentation:** Fragmentation is based on the predicate defined directly on that relation's own attributes (as in the example above).
- **Derived Horizontal Fragmentation:** Fragmentation of a relation is based on a predicate defined on **another (related) relation** — e.g. fragmenting ORDERS based on how CUSTOMER is fragmented.

(b) Vertical Fragmentation

Definition: The relation is divided **column-wise**. Each fragment contains a **subset of attributes (columns)** of the original table, but **all its rows**. The **primary key** must be included in every fragment so the original table can be reconstructed using a JOIN.

Example: A table *EMPLOYEE(EmpID, Name, Dept, Salary, Address)* can be vertically fragmented so that: Fragment 1 = (EmpID, Name, Dept) kept at the HR office; Fragment 2 = (EmpID, Salary, Address) kept at the Payroll office. Both fragments repeat EmpID (the key) to allow reconstruction.

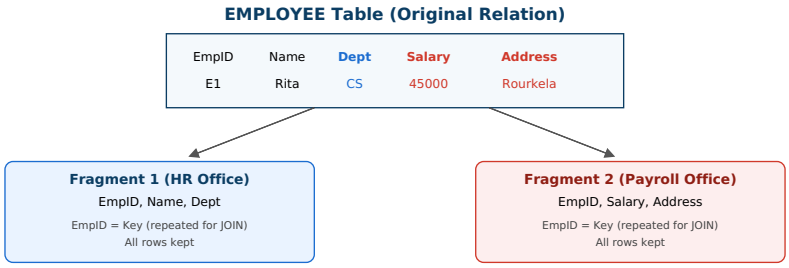


Fig 4: Vertical Fragmentation — columns are split into groups, each fragment keeps all rows but only some columns; the key column is repeated in every fragment.

(c) Hybrid (Mixed) Fragmentation

Definition: A combination of horizontal and vertical fragmentation applied one after another — e.g. a table is first fragmented horizontally, and then each horizontal fragment is further fragmented vertically (or vice versa).

Example: The STUDENT table is first split horizontally by City (Rourkela / Cuttack), and then each city fragment is further split vertically into (RollNo, Name) and (RollNo, Branch, City) for different departments to access.

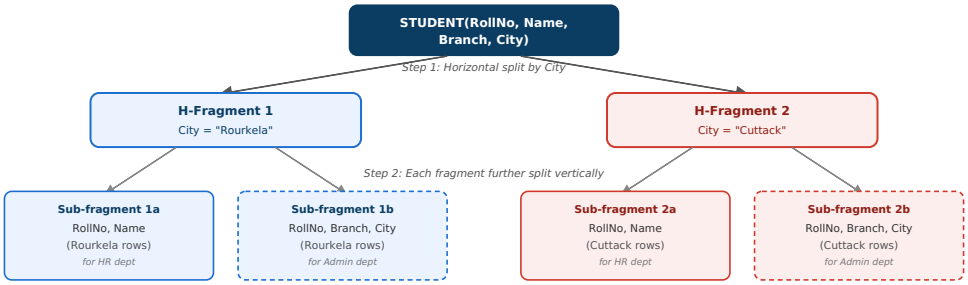


Fig 5: Hybrid (Mixed) Fragmentation — STUDENT is first split horizontally by City into two fragments, then each horizontal fragment is further split vertically into column subsets for different departments.

Basis	Horizontal Fragmentation	Vertical Fragmentation
Splits by	Rows (tuples)	Columns (attributes)
Each fragment has	All columns, subset of rows	All rows, subset of columns
Uses	Selection (σ) operation	Projection (π) operation
Reconstruction by	UNION of fragments	JOIN of fragments (on key)
Typical use case	Branch-wise data (bank branches)	Department-wise access to same records

Shortcut

Horizontal → rows → Selection (σ) → UNION. Vertical → columns → Projection (π) → JOIN. Just remember "H for rowS (Horizontal), V for column JOIN". And since Selection (σ) picks rows and Projection (π) picks columns, it's easy to link: **σ with Horizontal**, **π with Vertical**.

Common Mistake

Do not forget that in **vertical fragmentation** the primary key MUST be repeated in every fragment; students often wrongly assume vertical fragments must be fully disjoint like horizontal ones.

2. Allocation

Definition: Allocation is the process of **deciding at which site(s) each fragment (and its replicas, if any) will be physically stored**. It is essentially the "placement" decision made after fragmentation (and replication) has been designed, so that overall performance (speed, cost, availability) is optimized.

Allocation Strategies

Strategy	Description
Centralized	All fragments are designed, but placed back at just one single site.
Partitioned (Fragmented) Allocation	Each fragment is placed at exactly one site — no fragment is duplicated (matches "No Replication").
Replicated Allocation	Fragments are placed at more than one site as copies (matches full/partial replication).

Example: For a bank, the horizontal fragment "Accounts of Rourkela Branch" is **allocated** to the Rourkela server (its home site) because that is where it is used most often; a rarely changed "Bank Holiday List" fragment may be **allocated/replicated** to all branch servers for quick local access.

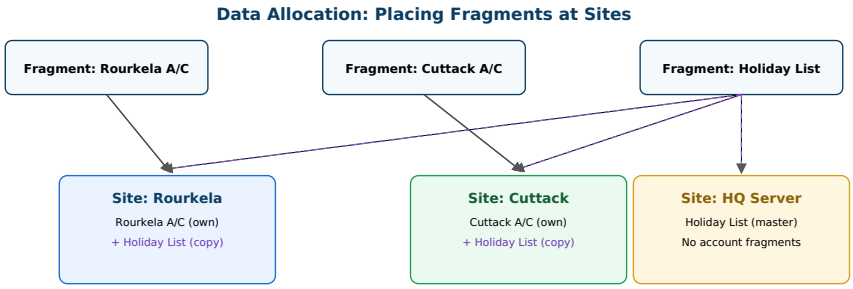


Fig 6: Allocation — the "home" fragment of each branch is placed (allocated) at its own site (solid arrows); a shared reference fragment is replicated/allocated to all sites (dashed arrows).

Remember
Allocation answers the question **"WHERE does each piece of data live?"** — it comes AFTER fragmentation decides "HOW is data divided?" and works together with replication which decides "HOW MANY copies exist?".

3. Replication

Definition: Replication means storing **identical copies** of the same data (a full table or a fragment) at **more than one site**. This improves availability and speeds up read access, since a copy is always close to the user, at the cost of extra storage and more work to keep copies updated.

Forms of Replication

Type	Description
Full Replication	A complete copy of the entire database is stored at every site.
Partial Replication	Only some fragments/tables (the frequently used ones) are copied to some sites, not all.
No Replication	Each fragment exists at exactly one site only (no duplication at all).

Example: A university's "Fee Structure" table (rarely changed, read very often) is **fully replicated** at every campus (Rourkela, Cuttack, Sambalpur) so that students anywhere can check fees instantly without contacting a remote server.

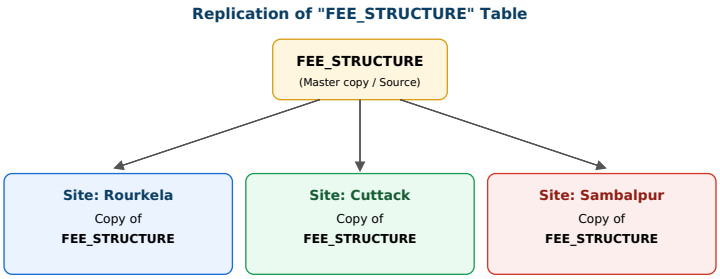


Fig 7: Replication — the same table is copied (duplicated) fully at every site so any site can read it locally.

Advantages of Replication	Disadvantages of Replication
High availability — data accessible even if one site fails	Extra storage space needed for copies
Faster read access (local copy)	Update overhead — every copy must be updated together
Reduces network traffic for read queries	Risk of copies becoming inconsistent if not synchronized

Exam Point
Replication mainly improves **read performance and availability** but makes **updates (write operations) costlier**, because every copy must be kept consistent.

Comparison: Fragmentation vs Allocation vs Replication

Aspect	Fragmentation	Allocation	Replication
Main Question	How to divide the data?	Where to place the data?	How many copies to keep?
Goal	Reduce unnecessary data at a site	Optimize placement for performance	Improve availability & read speed
Result	Smaller logical fragments	Physical site assignment	Duplicate copies of data
Extra storage needed?	No (data just split)	Depends on chosen strategy	Yes (copies take space)

Quick Revision / Cheat Sheet — Chapter 11.1

Key Definitions (one line each)

- **Distributed Database:** Logically one database, physically stored across multiple networked sites.
- **DDBMS:** Software managing a distributed database, hiding distribution from users.
- **Fragmentation:** Dividing a relation into smaller fragments (horizontal/vertical/hybrid).
- **Replication:** Storing copies of the same data at multiple sites.
- **Allocation:** Deciding which site(s) store which fragment/replica.

Characteristics of DDB (quick list)

- Data logically integrated, physically distributed
- Distribution transparency to users
- Local site autonomy
- No single central control point
- Connected via network; supports local + global transactions

Fragmentation Rules — "C R D"

- **C**ompleteness — no data lost
- **R**econstruction — original table can be rebuilt
- **D**isjointness — minimal duplication (except key in vertical)

Comparison Table

Technique	Splits	Rebuild Using
Horizontal Fragmentation	Rows	UNION
Vertical Fragmentation	Columns	JOIN
Hybrid Fragmentation	Rows + Columns	UNION + JOIN
Allocation	N/A (placement decision)	N/A
Replication	N/A (copies, not split)	N/A

Forms of Replication

- Full Replication — copy at every site
- Partial Replication — copy at some sites
- No Replication — one copy only, at one site

Allocation Strategies

- Centralized — all data at one site
- Partitioned — each fragment at exactly one site (no duplicates)
- Replicated — fragments copied to multiple sites

One-Line Exam Facts

- Horizontal fragmentation keeps ALL columns but SOME rows.
- Vertical fragmentation keeps ALL rows but SOME columns; key column repeats.
- Replication increases read speed & availability but raises update cost.
- Allocation decides the physical site location of a fragment/replica.
- Distribution transparency = user is unaware of where data physically resides.
- A distributed database is NOT the same as several unrelated databases.